Student Course Management

Spring Boot Application — SGA2 Project Documentation

Course	Big Data Analytics
Project	SGA2 — Spring Boot CRUD Application
Entities	Student and Course (ManyToMany)
Tech Stack	Spring Boot 2.7 / Spring MVC / Spring Data JPA / JSP / H2 / JUnit + Mockito
Operations	Create, Read, Update

1. Entity Relationship Design

The application manages two entities: **Student** and **Course**. The relationship between them is **ManyToMany** — a student can enroll in multiple courses, and each course can have many students. JPA manages this through a join table called **student_courses** with columns *student_id* and *course_id*.

Student is the owning side and holds the @JoinTable definition. Course uses mappedBy to point back to the Student side. Both sides use LAZY fetch to avoid unnecessary joins on every query.

Student Entity Fields

Field	Type	Constraint
id	Long	Primary Key, Auto-generated
firstName	String	Not blank
lastName	String	Not blank
email	String	Not blank, valid email, unique
age	Integer	Min 16, Max 60
courses	Set<Course>	ManyToMany, LAZY

Course Entity Fields

Field	Type	Constraint
id	Long	Primary Key, Auto-generated
title	String	Not blank, unique
description	String	Not blank, max 500 chars
credits	Integer	Min 1, Max 6
instructor	String	Not blank
students	Set<Student>	ManyToMany (mappedBy), LAZY

2. Database Setup and Sample Data

The application uses H2 in-memory database with **create-drop** DDL strategy. Tables are created automatically from entity definitions on startup. A DataInitializer component seeds the database with 10 courses and 12 students, each enrolled in 3 courses.

The initializer runs inside a single @Transactional method so all entities stay in the same JPA persistence context. Without this, saving students with already-persisted courses throws a detached entity error because each individual repository.save() call opens and closes its own transaction.

Sample data snippet:

```
Course c1 = courseRepository.save(new Course("Data Structures", "Covers arrays, linked
lists...", 4, "Dr. Alice Johnson"));
Student s1 = new Student("Arjun", "Sharma", "arjun.sharma@example.com", 20);
s1.getCourses().addAll(Arrays.asList(c1, c3, c5));
studentRepository.save(s1);
```

3. Create Operation

Both entities have an Add form accessible from the navigation. The student form includes checkboxes for course enrollment. On submission, bean validation runs first via @Valid, then the service checks for duplicate email (students) or duplicate title (courses) and rejects the field with an inline error message if found.

Controller method (student create):

```
@PostMapping("/add")
public String addStudent(@Valid @ModelAttribute("student") Student student,
                         BindingResult result,
                         @RequestParam(value="courseIds", required=false) List<Long>
courseIds,
                         Model model, RedirectAttributes redirectAttributes) {
    if (result.hasErrors()) {
        model.addAttribute("allCourses", courseService.findAll());
        return "student/add";
    }
    if (studentService.existsByEmail(student.getEmail())) {
        result.rejectValue("email", "error.student", "A student with this email already
exists");
        model.addAttribute("allCourses", courseService.findAll());
        return "student/add";
    }
    // resolve course IDs to managed entities, then save
    studentService.save(student);
    redirectAttributes.addFlashAttribute("successMessage", "Student added
successfully!");
    return "redirect:/students";
}
```

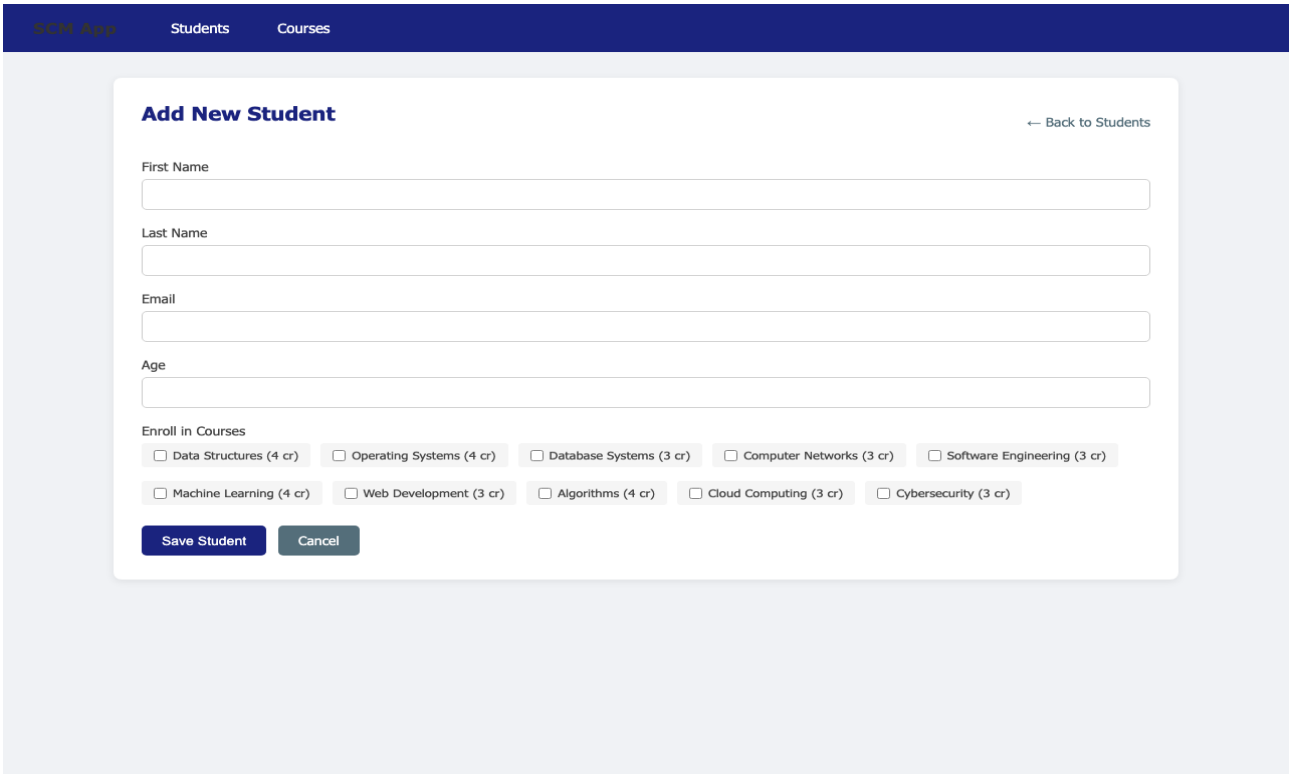

Figure 1 — Add Student form with course enrollment checkboxes

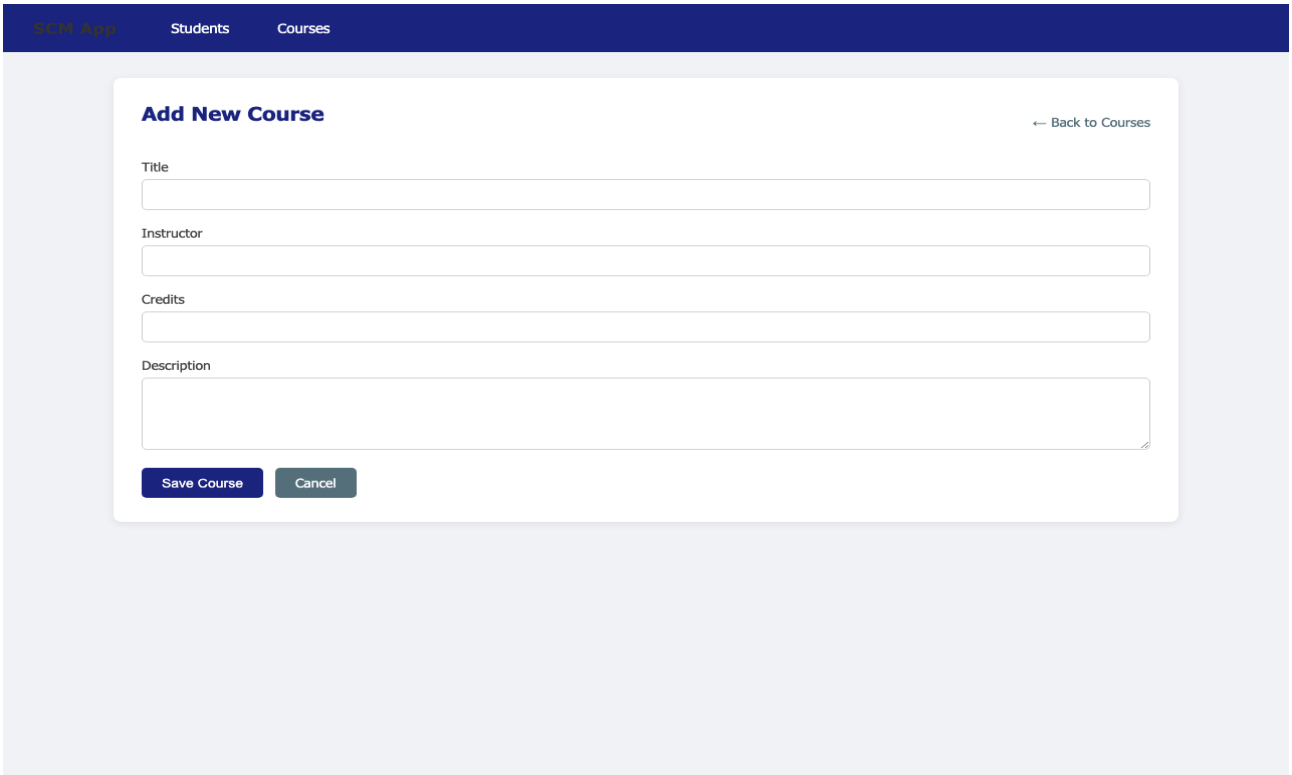

Figure 2 — Add Course form

4. Read Operation

The students list page renders two tables. The first lists all students with their enrolled courses shown as tags. The second table is the join view, powered by a JPQL inner join query using a DTO projection.

The courses list page shows each course with a clickable enrollment count. Clicking it runs a second join query to display only the students enrolled in that specific course.

Custom JPQL join queries:

```
@Query("SELECT new com.example.studentcourse.dto.StudentCourseDTO("
     + "s.firstName, s.lastName, c.title, c.instructor) "
     + "FROM Student s JOIN s.courses c ORDER BY s.lastName, s.firstName")
List<StudentCourseDTO> findAllEnrollments();

@Query("SELECT s FROM Student s JOIN s.courses c WHERE c.id = :courseId")
List<Student> findStudentsByCourseId(@Param("courseId") Long courseId);

@Query("SELECT c FROM Course c JOIN c.students s WHERE s.id = :studentId")
List<Course> findCoursesByStudentId(@Param("studentId") Long studentId);
```

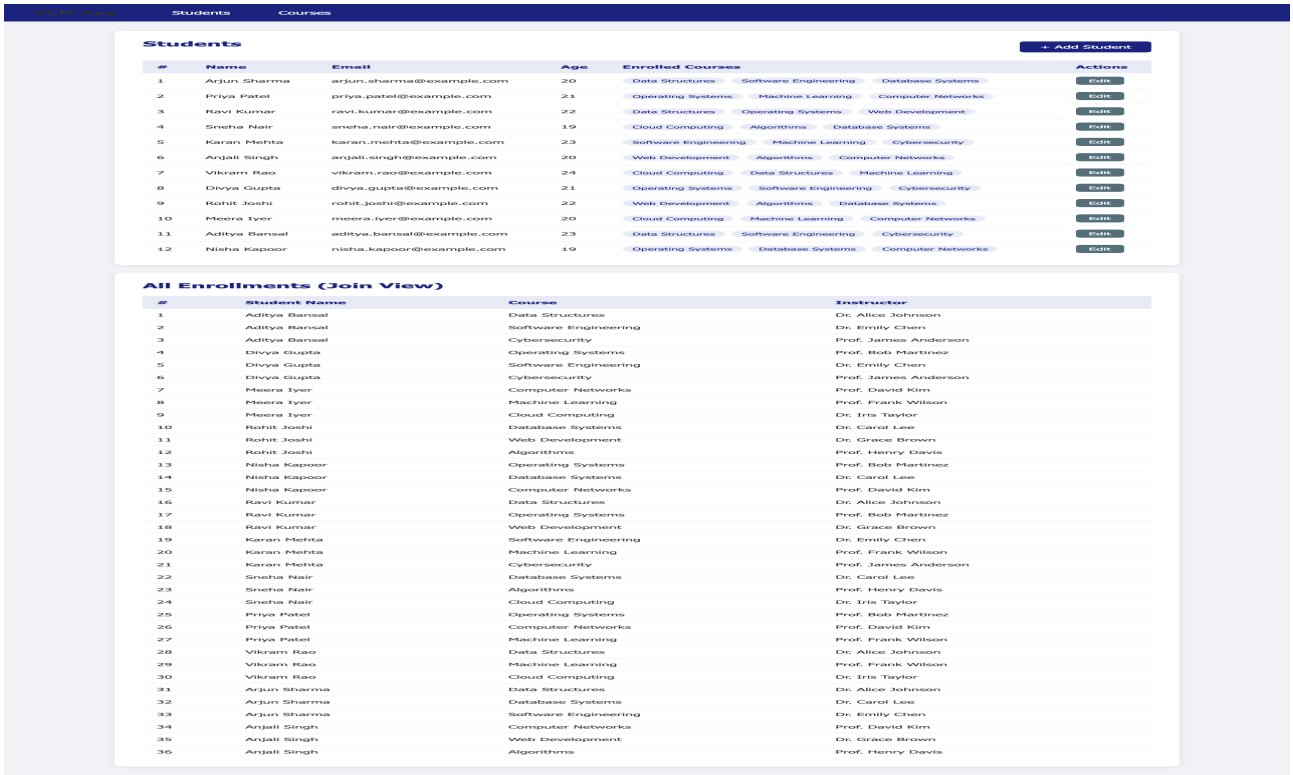

Figure 3 — Students list with enrolled course tags and full enrollment join table

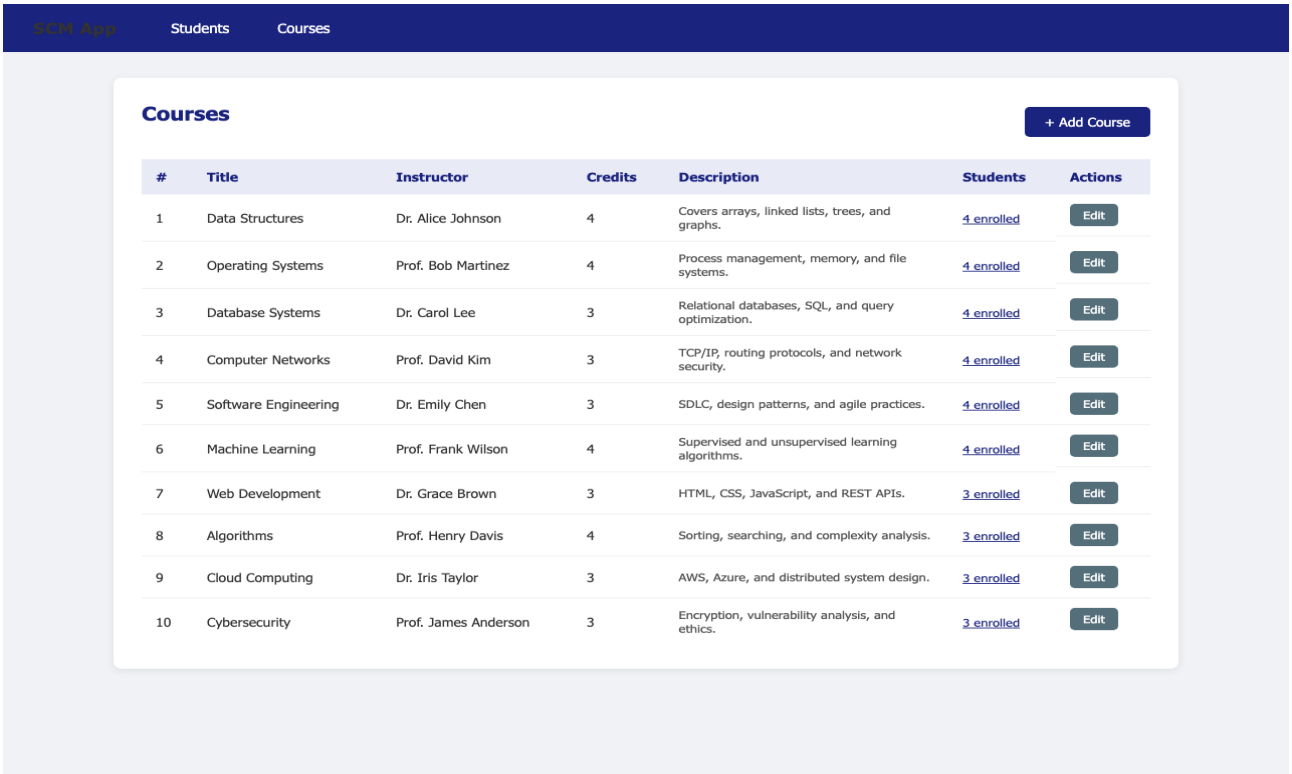

Figure 4 — Courses list with enrollment counts

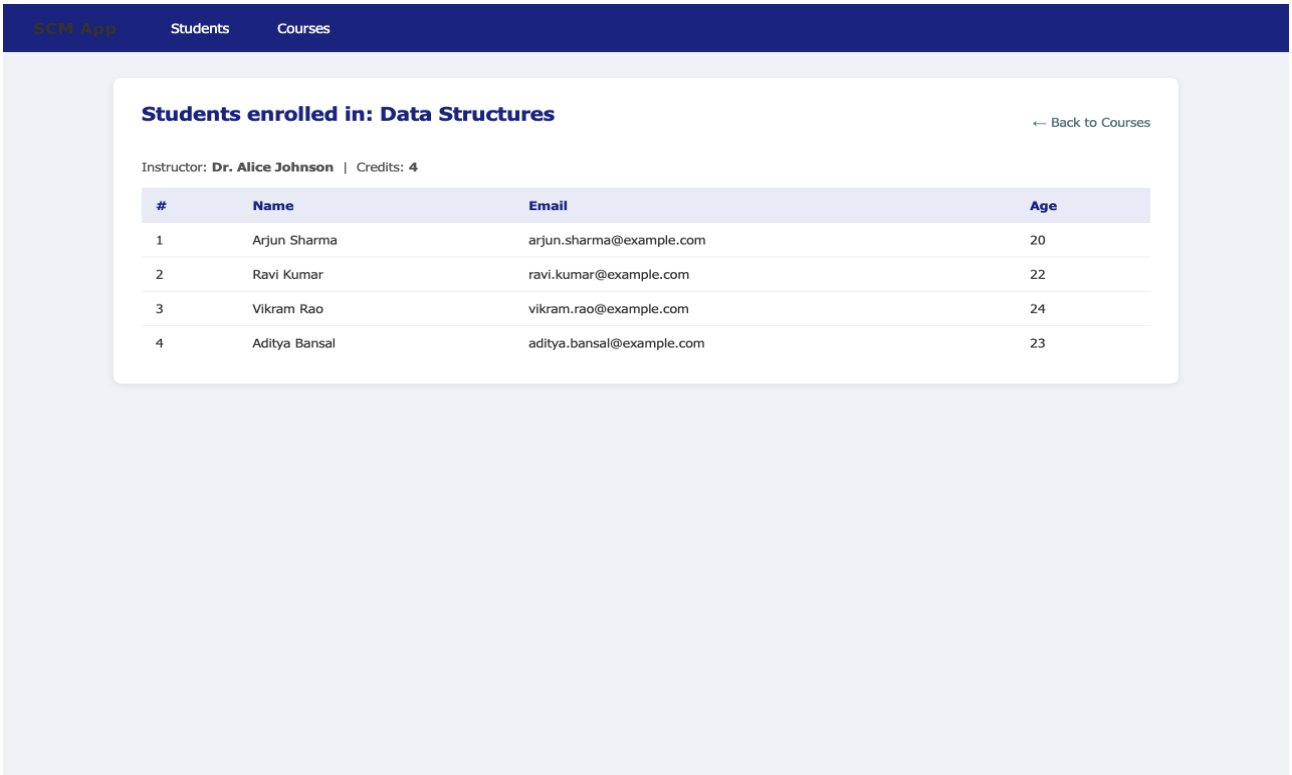

Figure 5 — Students enrolled in a specific course (join query result)

5. Update Operation

Clicking Edit on any student or course loads a pre-populated form. For students, the enrolled courses are pre-checked in the checkbox list using a JSTL comparison loop. On POST, the service reloads the entity fresh from the database before applying changes — this avoids stale state and ensures changes are applied to a managed entity.

Service update method:

```
public Student update(Student student) {
    Student existing = studentRepository.findByIdWithCourses(student.getId())
        .orElseThrow(() -> new IllegalArgumentException("Student not found"));
    existing.setFirstName(student.getFirstName());
    existing.setLastName(student.getLastName());
    existing.setEmail(student.getEmail());
    existing.setAge(student.getAge());
    Set<Course> managedCourses = new HashSet<>();
    for (Course c : student.getCourses()) {
        courseRepository.findById(c.getId()).ifPresent(managedCourses::add);
    }
    existing.setCourses(managedCourses);
    return studentRepository.save(existing);
}
```

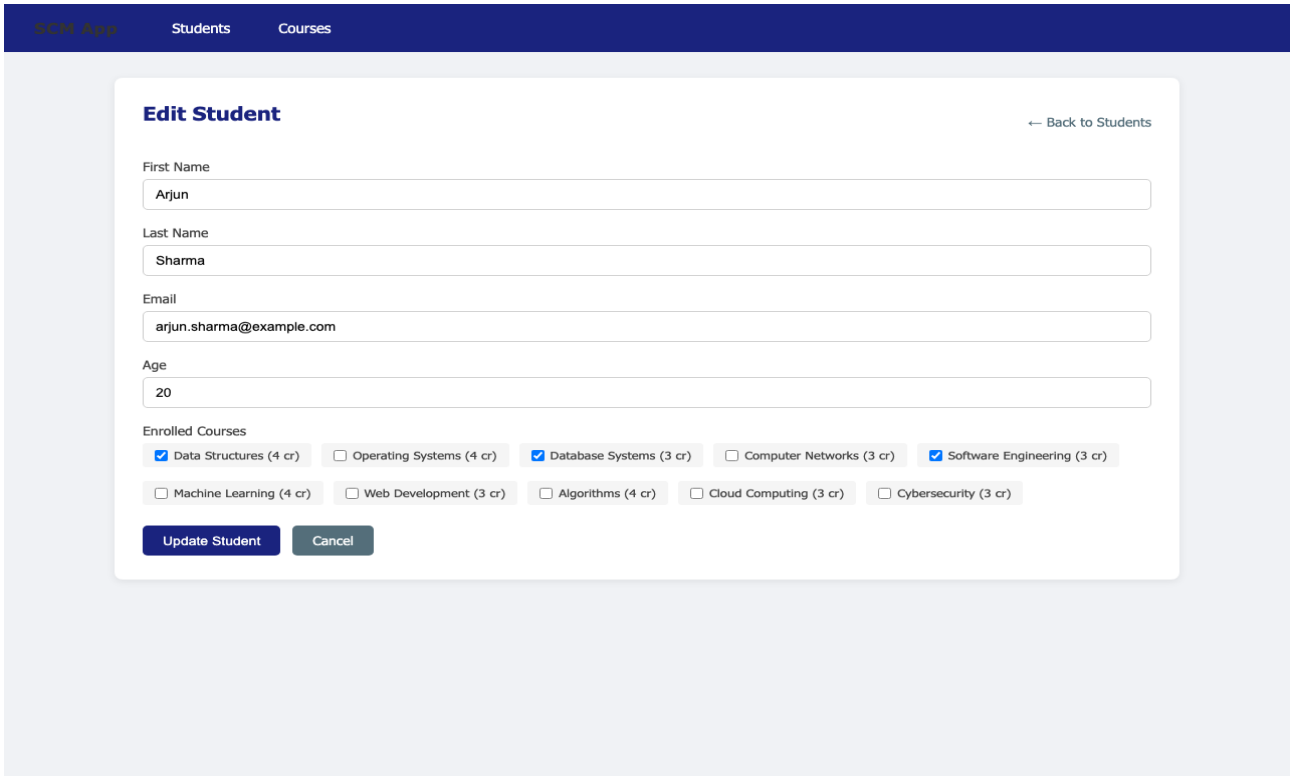

Figure 6 — Edit Student form with pre-checked enrolled courses

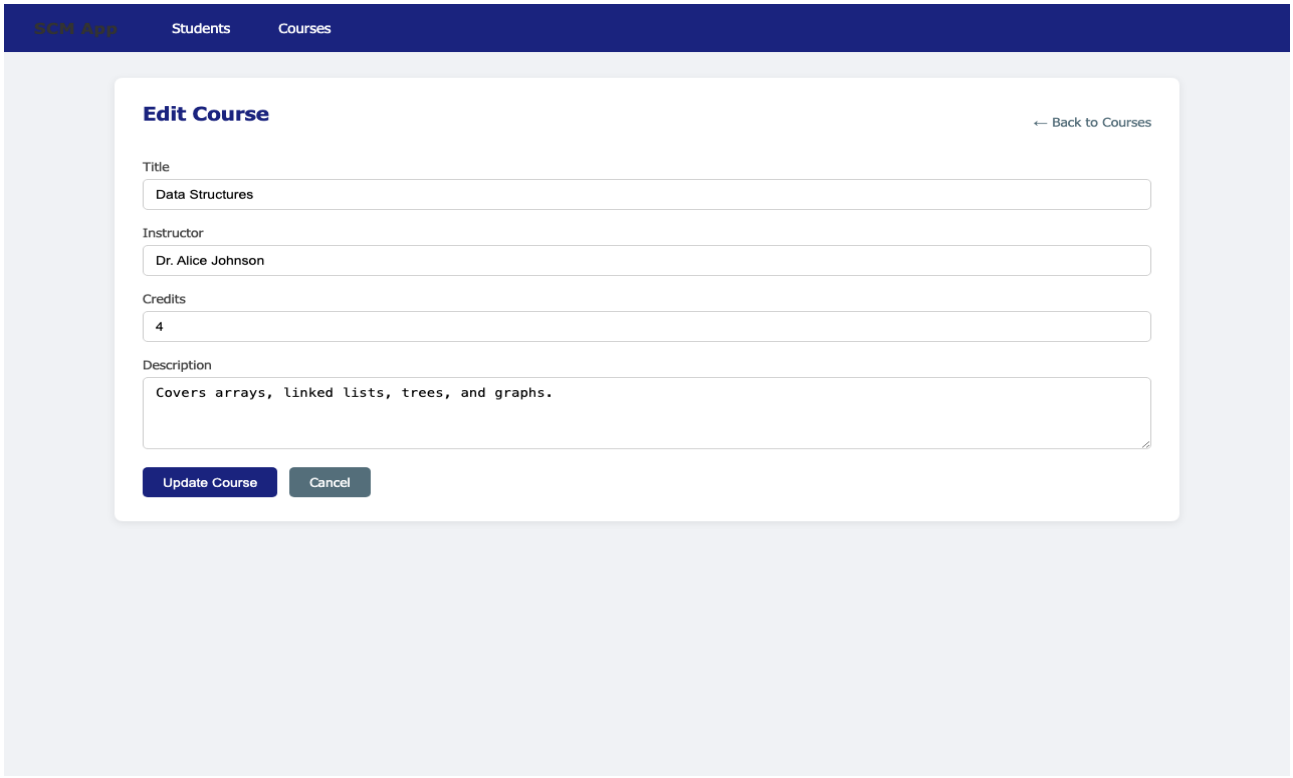

Figure 7 — Edit Course form

6. Architecture Overview

Layer responsibilities:

Layer	Classes	Responsibility
Repository	StudentRepository CourseRepository	JPA queries, custom JPQL joins, DTO projections
Service	StudentServiceImpl CourseServiceImpl	Business logic, duplicate checks, entity resolution
Controller	StudentController CourseController	HTTP request handling, model binding, redirect logic
View	7 JSP pages	Form rendering, data display, CSS styling
Config	DataInitializer	Seed 12 students + 10 courses on startup

Project File Structure

```
src/main/java/com/example/studentcourse/
    StudentCourseApplication.java
    config/DataInitializer.java
    entity/Student.java, Course.java
    dto/StudentCourseDTO.java
    repository/StudentRepository.java, CourseRepository.java
    service/StudentService.java, StudentServiceImpl.java
             CourseService.java, CourseServiceImpl.java
    controller/HomeController.java, StudentController.java, CourseController.java
src/main/webapp/WEB-INF/views/
    common/header.jsp, footer.jsp
    student/list.jsp, add.jsp, edit.jsp
    course/list.jsp, add.jsp, edit.jsp, students.jsp
src/test/java/com/example/studentcourse/
    repository/StudentRepositoryTest.java, CourseRepositoryTest.java
    service/StudentServiceTest.java, CourseServiceTest.java
```

7. Testing

33 unit tests across 4 test classes. All pass. No manual setup required.

Test breakdown:

Test Class	Type	Tests	What is covered
StudentRepositoryTest	@DataJpaTest (real H2)	7	findByEmail, join query, DTO projection, findByIdWithCourses, save
CourseRepositoryTest	@DataJpaTest (real H2)	6	findByTitle, findByInstructor, join query, save
StudentServiceTest	Mockito	10	findAll, findById, save, update, duplicate checks, DTO list
CourseServiceTest	Mockito	10	findAll, findById, save, update, duplicate checks, same-ID check

```
mvn test

[INFO] Tests run: 33, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

8. Challenges and Solutions

Detached Entity Error on Startup

When the DataInitializer saved students with already-persisted Course objects, Hibernate threw a PersistentObjectException because the courses were detached — each previous courseRepository.save() had closed its own transaction. The fix was adding @Transactional to the run() method so all entities stay in the same persistence context throughout the entire seed operation.

Pre-checking Enrolled Courses in Edit Form

Spring's form:checkboxes tag does not bind well to ManyToMany collections when the form field name (courseIds) differs from the entity field (courses). The solution was using plain HTML checkboxes with a nested JSTL c:forEach loop that compares each available course ID against the student's current courses set and marks the matching ones as checked.

Duplicate Validation vs Database Constraint Exceptions

Relying on the database unique constraint to catch duplicates surfaces a 500 error page to the user. Instead, the service layer checks for existing email or title before saving and calls result.rejectValue() to attach the error directly to the form field, giving the user a clean inline message.

ManyToMany Enrollment Update

Updating a student's course enrollments required resolving the submitted course IDs back into managed JPA entities before setting them on the student. Passing raw Course objects with only an ID set would cause Hibernate to treat them as transient or detached. Each ID is fetched via courseRepository.findById() to get a properly managed entity before the set is applied.

9. Setup and Run Instructions

Prerequisites

Java 11+, Maven 3.6+. No database installation needed — H2 runs in memory.

Run the application

```
git clone <github-url>
cd student-course-app
mvn spring-boot:run
```

Then open http://localhost:8080 in your browser.

Run tests

```
mvn test
```

H2 Console

While the app is running, the H2 console is available at http://localhost:8080/h2-console with JDBC URL jdbc:h2:mem:studentcoursedb and no password.

GitHub Repository

GitHub URL: https://github.com/jeeevan11/student-course-app